Reconnaissance de son dans la musique

1 Contexte

Maintenant que l'on a vu la convolution et PyTorch, on va se lancer dans un projet de reconnaissance automatique d'instruments de musique dans un fichier audio.

Ce que l'on vise : on veut un modèle capable d'identifier, pour au moins 70% des morceaux en test, un instrument parmi ceux présents. Pour cela, vous serez libres (1) de la représentation du son, (2) de l'architecture du modèle (en restant dans les limites du cours, et en utilisant de la convolution) et (3) des paramètres de la procédure d'entraînement.

Ce dont on dispose : un ensemble de données d'entraînement et d'un ensemble de données de test. Il s'agit d'une version simplifiée du dataset IRMAS, comportant initialement 11 instruments différents. Pour simplifier le problème, on va se contenter des instruments suivants : la guitare accoustique (gac), l'orgue (org), le piano (pia) et la voix humaine (voi). Ensuite :

- l'ensemble d'entraînement est composé de quatre sous-dossiers, chacun comportant un certain nombre de fichiers .wav de 3 secondes, dont l'instrument dominant est celui nommant le dossier ;
- l'ensemble de données de test contient des morceaux de chansons (également en .wav) de longueurs variables, associés à un fichier contenant la liste des instruments présents parmi les 11 du dataset initial.

On garantit le fait que, pour chaque instance de test : (1) au moins un des quatre instruments qui nous interessent se trouve dans cette liste et (2) les proportions des instruments sont cohérentes entre l'ensemble d'entraînement et l'ensemble de test (à peu près équilibré).

Remarques : ce projet a été initié par une étudiante de 5ème année, Daryna Lotetska, bureau A79, n'hésitez pas à venir lui poser toutes vos questions.

2 Cahier des charges

On attend de vous la *pipeline* suivante :

1. transformation des données ;
2. entraînement d'un modèle et sélection du meilleur via validation ;
3. évaluation du modèle obtenu sur l'ensemble de test, via une métrique spécifique (pour chaque instance, 1 si le modèle détecte un instrument présent, 0 sinon) ;
4. bonus : une application exploitant votre modèle (exemple : lecteur audio minimal avec affichage de l'instrument détecté).

3 Trucs et astuces

3.1 La représentation du son

Avant tout, il va vous falloir choisir une représentation du son, parmi plusieurs possibles :

- une transformation en spectrogramme 2D est un grand classique pour la visualisation du son ;
- des transformations 1D multi-canaux sont également envisageables.

Faites des recherches, et réunissez les avantages et incovénients des alternatives avant l'aller plus loin.

3.2 Gestion de la taille des entrées

Pour l'instant, vous avez toujours aborder des problèmes avec des tailles d'entrées fixes (par exemple, les instances MNIST sont définies par 784 caractéristiques). Ça ne sera pas le cas ici. Vous aurez alors deux solutions :

1. la première est de définir une taille d'entrée commune, et de redimensionner l'entrée si elle est trop petite (ajouter des zéros à la fin) ou si elle est trop grande (découper l'entrée en morceaux) ;
2. la deuxième solution exploite l'avantage des couches de convolution et de pooling est d'être insensible à la taille de l'entrée, le seul problème résidant dans le linkage entre la partie convolution du modèle, et la partie dense qui s'attend à une taille fixe donnée.

Pour la deuxième solution, vous pouvez utiliser une couche de pooling adaptatif, parmi `AdaptiveAvgPool1d`, `AdaptiveAvgPool2d`, `AdaptiveMaxPool1d` et `AdaptiveMaxPool2d`. Ces couches fonctionnent comme celles que vous avez déjà rencontrées, à l'exception qu'elles fixent la taille de la matrice de sortie. Par exemple, `AdaptiveAvgPool2d((8,8))` prendra un tenseur de dimension (C, N, N), et renverra un tenseur de dimension $(C, 8, 8)$, rempli avec les moyennes des 64 zones du tenseur original, sur le même nombre de canaux. Cette couche pourra être utilisée juste avant l'applatissement et le passe sur la partie dense du modèle.

Le fait d'avoir des instances de tailles variables oblige à fixer la taille de batch à 1, car on ne peut pas empiler des tenseurs de taille différentes. Dans notre cas, cela n'arrive que pour l'ensemble de test, ce qui n'est pas génant, mais oblige à traiter les instances une par une, contrairement à l'ensemble d'entraînement qui pourra se faire en lot.

3.3 Combattre le sur-apprentissage

Vous allez évidemment diviser votre ensemble d'entraînement complet en un sous-ensemble d'entraînement et un sous-ensemble de validation (on conseillera 75% et 25%), et vous risquez de remarquer que votre modèle a fortemment tendance à surapprendre. Afin de contrer cela, on pourra :

- minimiser la taille du modèle : plus le modèle contient de paramètres, plus il risque d'apprendre par coeur ;
- utiliser la normalisation par batch, via les couches `BatchNorm1d` ou `BatchNorm2d`, en fonction de vos architectures, à mettre en place entre les couches de convolution et les activations ;
- utiliser du `dropout` : cela permet de désactiver aléatoirement des neurones, et favorise la généralisation ;
- mettre en place une augmentation des données avec *SpecAugment* : lors de l'entraînement, on désactive aléatoirement des canaux ou des dimensions temporelles de l'entrée, pour simuler des pertes de signal.

3.4 Minimiser le temps d'apprentissage

En fonction de votre configuration et choix d'implémentation, le temps d'apprentissage peut être long. Voici quelques pistes pour vous aider à améliorer cela :

- augmenter la taille de batch (et le taux d'apprentissage) si vous disposez d'un GPU ;
- désactiver la mise à jour du gradient lors de l'évaluation du modèle sur l'ensemble de validation via `modele.eval()` et `torch.no_grad()` ;
- évitez les traitements au chargement du batch : si vous devez appliquer une transformation aux données, faites le avant l'apprentissage.

4 Conditions de rendu

Le travail est à réaliser par groupe de 4 maximum. Le projet est à rendre pour le [A DEFINIR] à 18h, via Moodle, dans une archive .zip contenant :

- l'ensemble des fichiers .py nécessaires au fonctionnement de la *pipeline* ;
- un fichier README.txt contenant les noms des participants, une description de votre travail, et une mention concernant l'utilisation de l'IA générative dans votre projet (si vous l'avez utilisée, indiquez dans quel but et les sections de code concernées).